CYBER FORENSICS CAPSTONE

Problem 31 · Mobile App Usage Timeline Builder

Fuseline

Cyber Forensics Capstone Project

Project report

Prepared by

Navneet Nanda	CH.SC.U4CYS24039
Roshan Srikanth	CH.SC.U4CYS24057

Assigned task

Correlate location data, browsing history, and app usage logs into a unified forensic timeline.

Suggested tools in the brief: Python, SQLite, Plaso.
Structure: acquisition, analysis, validation, report.
Software version described: 1.1.0.

Contents

1. Solution required 2
2. Solution proposed 2
3. Algorithm 4
4. Outcome 5
5. Conclusion 6
6. Inference 7
7. References 7

This document describes Fuseline version 1.1.0 as implemented in the project repository. It is the project report. It is not the per-case evidence report the tool exports after a case is built.

1. Solution required

The assigned problem, from the Cyber Forensics Capstone brief (problem 31, Mobile App Usage Timeline Builder), is to correlate location data, browsing history, and app usage logs into one forensic timeline. The suggested path is to extract timestamps from those artifacts, put them on one time base, relate events that sit together in time, and present that view. The named technology is Python, SQLite, and Plaso. The work is organised as acquisition, analysis, validation, and report.

The practical difficulty is that the three sources do not arrive as one log. A location file may be a GPS track, a GPX file, or a Google location export. Browsing may be a Chromium History database or a Firefox places database. App usage may be an Android usage-stats database, a CSV or XML export, or the text printed by `dumpsys usagestats`. The clocks differ. Some values are UTC. Some carry an offset. Some are local time with no zone. Sorting the raw strings does not produce a reliable order.

What the brief asks for is a chronological view an examiner can review, with each point traceable to the artifact it came from. NIST's mobile forensics guide and the SWGDE mobile collection guide treat acquisition and later examination as separate steps. Fuseline follows that split: collect the Android artifacts, then build the timeline from what was collected.

2. Solution proposed

Fuseline is a local program for Android artifacts. The examiner creates a case, records who is examining it, and sets the device timezone. Evidence is added by uploading files, or by an authorised USB debugging connection. A Python API does the parsing and storage. Each case is its own SQLite database. The browser interface is served with that API.

Figure 1. Path from evidence to the examiner's view.

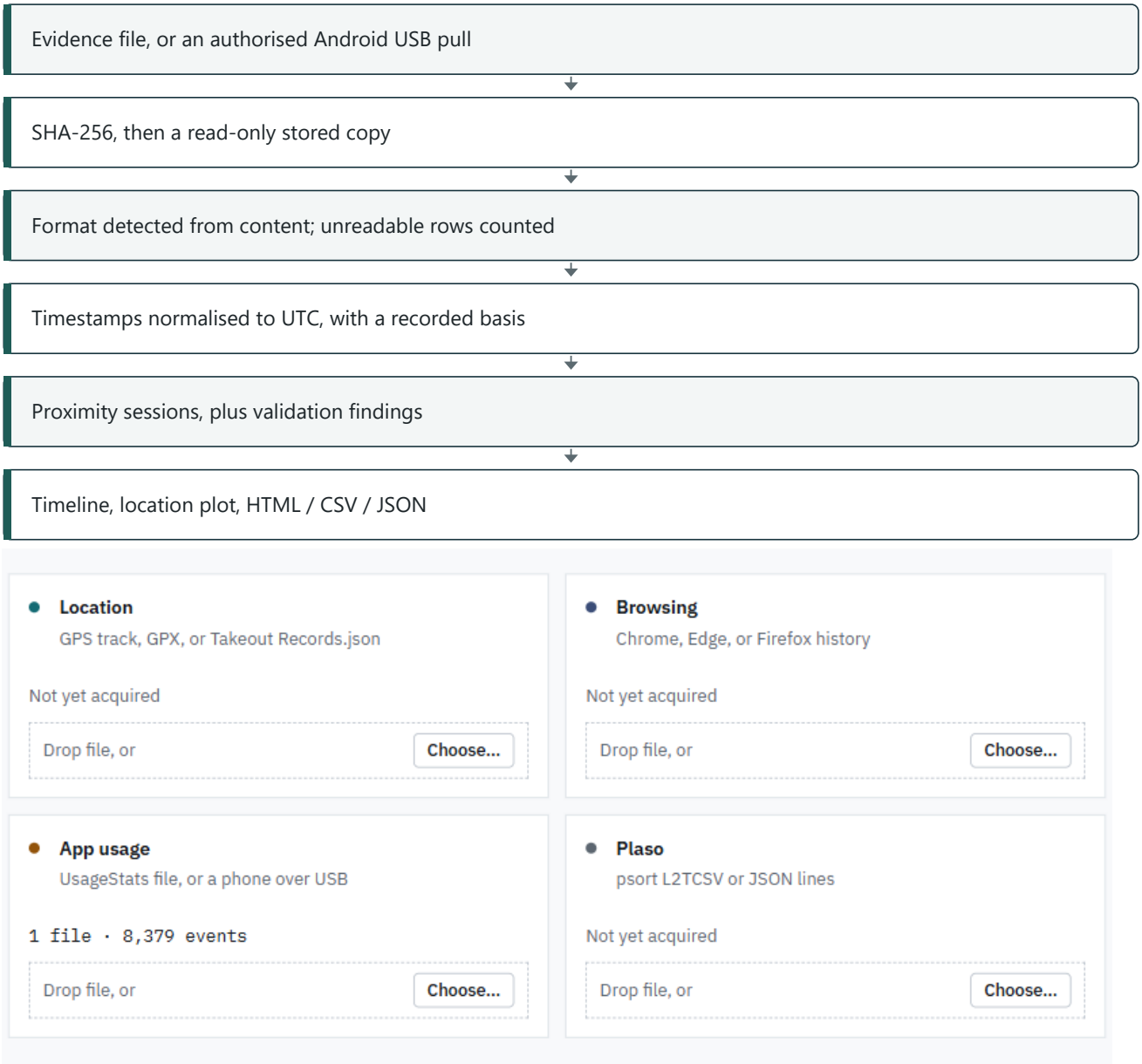


Figure 2. Acquisition screen. Each source is recognised from the file contents. This case has an Android usage-stats pull.

The parser is chosen by reading the file, not by trusting the file name. The formats recognised are listed below. Plaso is optional. If an examiner already has a Plaso storage file, `psort` can write L2TCSV or JSON lines, and Fuseline maps rows it understands onto the location, browsing, and app-usage lanes.

Source	Accepted input
Browsing	Chromium, Chrome, or Edge History; Firefox or Firefox for Android places.sqlite
App usage	Android UsageStats SQLite, CSV, or XML; adb dumpsys usagestats text
Location	CSV or SQLite with coordinates and time; GPX; Google Takeout Records.json; adb dumpsys location text when the dump contains timed fixes
Plaso	psort L2TCSV or JSON lines, optional

On an authorised USB debugging connection, Fuseline pulls `dumpsys usagestats`, a `dumpsys location` snapshot when that text contains timed coordinates, and supported files in shared storage (Download and Documents, including a nested Takeout folder). Browser history and other exports can be placed in those folders, or uploaded from the examiner's machine.

The API accepts requests for the host names it has been configured to allow. The default is loopback, which suits an examiner workstation. Case files and evidence copies stay on the machine that runs the API.

Usage-stats event numbers follow the Android `UsageEvents` constants. The CSV layout is the one used by common Android artifact parsers, including ALEAPP. Plaso is used as an import format, not as a library embedded in the program. No third-party forensic engine is shipped inside the repository.

3. Algorithm

3.1 Ingest

Each uploaded file is hashed with SHA-256, as specified in FIPS 180-4. The bytes are copied into the case store, the copy is hashed again, and the copy is made read-only. If that digest is already in the case, the stored copy is kept and the file is not parsed again. Rows that cannot be interpreted are skipped and counted, with a short reason stored on the artifact.

3.2 Time

Every event stores the UTC time used for sorting, the original source value, and a basis. Sorting and correlation use the UTC time. The basis stays on the event so the examiner can see which times were inferred. RFC 3339 is the profile used for the UTC timestamps the tool writes.

Basis	Meaning
absolute	The source was an epoch number, or a string already marked as UTC.
offset	The source carried an explicit offset, such as -05:00, and was converted.
assumed	The source had no zone. It was read in the case timezone and flagged.

When a timestamp has no zone, it is read in the case timezone and the basis is recorded as assumed. The validation step records `TZ_ASSUMED` for those events.

3.3 Proximity sessions

Events are ordered by UTC time and then by event identifier. An event joins the current cluster when its time is within the window of the immediately previous event. The default window is 300 seconds. This is single linkage along that order. Each event is in at most one cluster. Starting the scan from a different record does not change the clusters, because the sort is fixed.

A cluster is kept when it contains at least two distinct sources. The default sources that count are location, browsing, app usage, and a Plaso row that has been mapped onto one of those lanes. A cluster whose first and last events are more than 30 minutes apart is cut at the widest gap between neighbours. If two gaps are equal, the cut nearer the middle of the cluster is used. The pieces are tested again against the same rules.

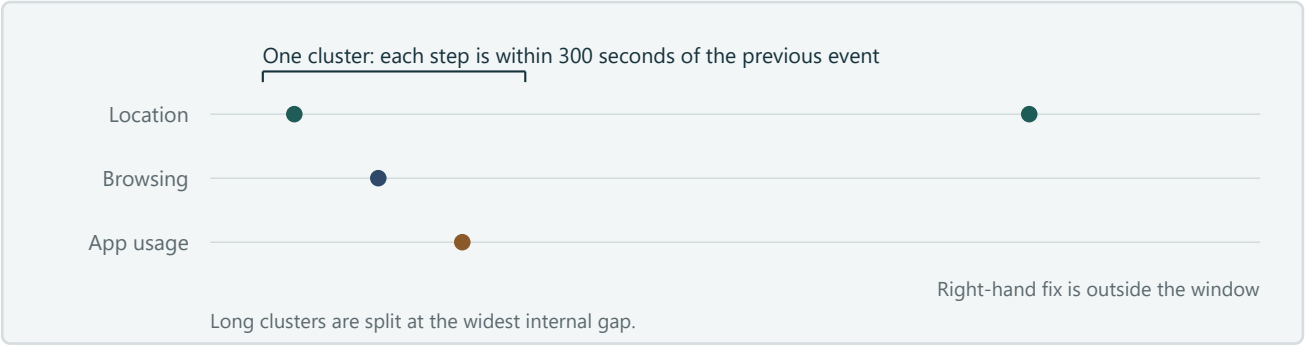


Figure 3. Diagram of the default rule. Two sources inside 300 seconds form a session. A run longer than 30 minutes is split.

The session identifier is a UUID version 5 computed from the member event identifiers, in order. The same evidence therefore produces the same session identifier. An event identifier is itself a UUID version 5 of the artifact SHA-256 and the row index inside that artifact.

The score used to rank sessions is the number of distinct sources, plus 0.5 if a location source is present, plus the smaller of 1 and (event count / 10). Sessions are ordered by that score.

If the cluster contains coordinates, the session stores their centroid and the radius of those points about the centroid. The radius is the spread of the points in that session.

Parameter	Default	Role
Window	300 seconds	Maximum gap between one event and the next inside a cluster
Minimum sources	2	A cluster with fewer sources is discarded
Maximum length	30 minutes	Longer clusters are split at the widest pause

3.4 Validation

After ingest, the case is checked and each check is stored as a finding. The codes below are the ones the program actually emits.

Code	When it appears
NO_EVENTS / EVENTS_PRESENT	The case has no normalised events, or it has some.
SINGLE_SOURCE / MULTI_SOURCE / NO_SOURCES	How many source types were stored.
DUPLICATE_HASH / HASH_INVENTORY	Whether two artifacts share a SHA-256, or each hash is recorded.
ROWS_SKIPPED	One or more source rows could not be parsed.
TZ_ASSUMED	Some timestamps had no zone and were read in the case timezone.
TIME_SPAN	The duration from the earliest event to the latest.
LARGE_GAP / GAP_OK	The largest gap between neighbouring events is over 7 days, or not.
NO_TIME_OVERLAP	Two sources never occupy the same time range, so they cannot meet in a session.
SESSIONS_FOUND / NO_SESSIONS	Whether any cluster met the source rule.
FUTURE_TS	Events dated more than one day ahead of the clock used at validation.
EARLY_TS	Events dated before 1 January 2008, often a zero or epoch placeholder.

3.5 Chain of custody

An append-only audit log records case creation, ingest, device pulls, session rebuilds, and report exports. Each entry includes the hash of the previous entry. The verify action re-hashes every stored evidence file and walks that chain. A match shows that the stored copy still equals the bytes that were ingested.

4. Outcome

Given artifacts the parsers recognise, the examiner gets one UTC timeline. The hour chart shows how many events fell in each hour, and which apps and record types those events are. The event list sits under that chart. Its filter searches title, package, URL, and domain. Selecting a row shows the original

timestamp, the time basis, the source file, and the SHA-256 of the stored copy.

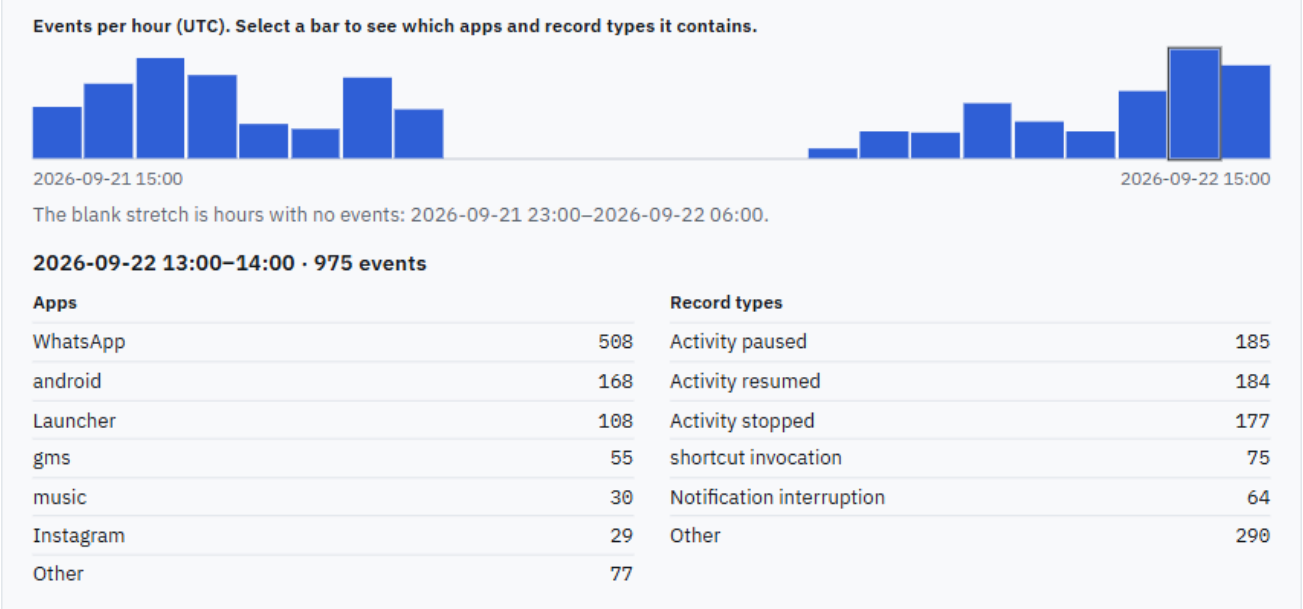


Figure 4. Hour chart for an Android usage-stats pull. The blank stretch is hours with no events. Selecting the busiest hour lists the apps and record types inside it.

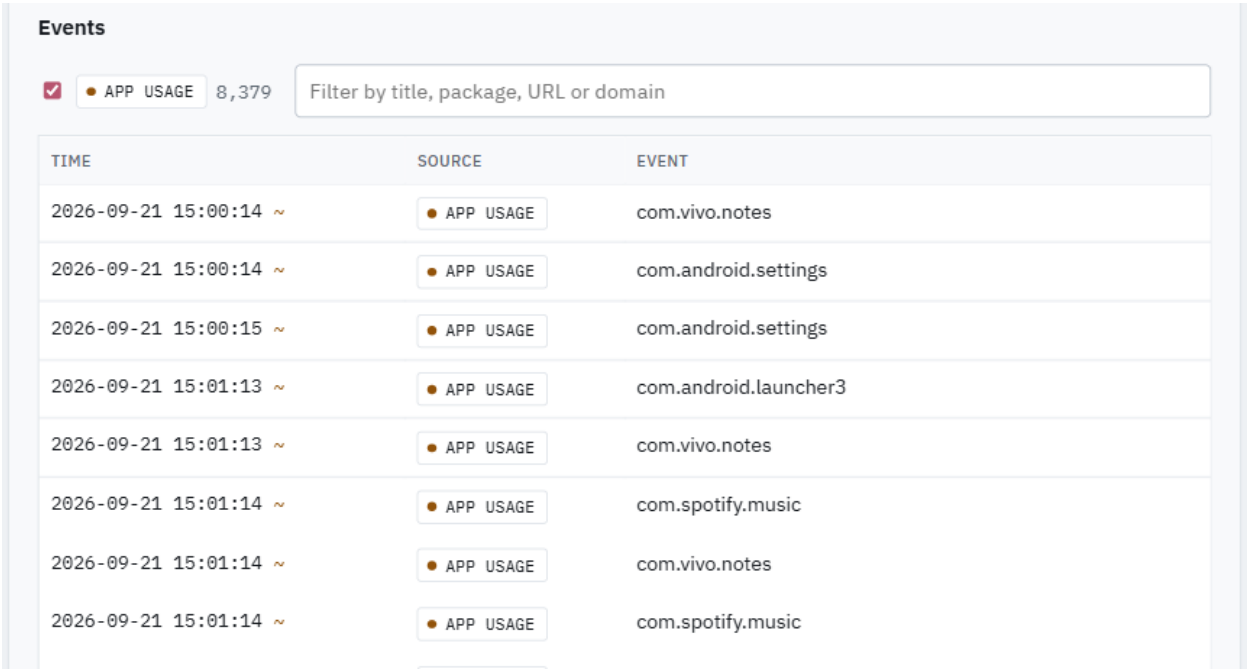


Figure 5. Event list. The filter is on this table. A tilde marks a timestamp that had no zone and was read in the case timezone.

A session is built only when events from at least two sources fall inside the window in section 3. A map is drawn only when location fixes include coordinates. A case that contains only app usage therefore has an hour chart and an event list, and no session list and no map.

This project report is separate from the case export. From a case the examiner can export HTML, CSV, JSON, or a case PDF. Exporting writes an audit entry that records the kind of export and the SHA-256 of the exported text.

The repository contains automated tests for the parsers, the session rules, the API, and the interface.

5. Conclusion

Fuseline carries out the assigned task. It takes location, browsing, and app-usage artifacts, places their timestamps on UTC, and shows where those sources fall close together in time. Acquisition, analysis,

validation, and reporting are separate steps. Each event can be traced to a stored file and a hash.

The correlation is a time window with an explicit length and an explicit rule for splitting long runs. Changing the window, the minimum number of sources, or the maximum length changes which sessions appear, and the examiner can see the values in use. The result is repeatable for the same bytes and the same settings.

The USB path collects Android usage statistics, a location snapshot when timed fixes are present, and files the examiner has placed in shared storage.

6. Inference

A session means that events from at least two sources lie inside the configured window, after the long-cluster split. The examiner can state that these records are close in time, which files they came from, and how each clock was read.

`LARGE_GAP` records a gap of more than seven days between neighbouring events. `NO_TIME_OVERLAP` records two sources whose time ranges do not meet.

Times marked assumed were read in the case timezone. The TZ_ASSUMED finding records that on the case.

A successful integrity check means the stored copies and the audit chain still match what the tool wrote. A location radius is the spread of the coordinates inside that session. A large radius means those points are far apart. Where a source file includes its own accuracy figure, that figure stays on the individual event.

7. References

The course problem statement is an internal brief, so it is described in section 1 and is not listed here. The works below are the public sources used for the methods in this report.

1. Ayers, R., Brothers, S., and Jansen, W. (2014). Guidelines on Mobile Device Forensics. NIST Special Publication 800-101 Revision 1.
   https://csrc.nist.gov/pubs/sp/800/101/r1/final

2. NIST (2015). Secure Hash Standard (SHS). FIPS 180-4.
   https://csrc.nist.gov/pubs/fips/180-4/upd1/final

3. Scientific Working Group on Digital Evidence (2025). Best Practices for Mobile Device Evidence Collection & Preservation, Handling, and Acquisition. SWGDE 18-F-003-2.0.
   https://www.swgde.org/wp-content/uploads/2025/09/2025-08-21-Best-Practices-for-Mobile-Device-Evidence-Collection-Preservation-Handling-and-Acquisition-18-F-003-2.0.pdf

4. Scientific Working Group on Digital Evidence (2026). Best Practices for Mobile Device Forensic Analysis. SWGDE 20-F-005-2.0.
   https://www.swgde.org/wp-content/uploads/2026/06/2026-06-23-Best-Practices-for-Mobile-Device-Forensic-Analysis-20-F-005-2.0.pdf

5. Klyne, G., and Newman, C. (2002). Date and Time on the Internet: Timestamps. RFC 3339.
   https://www.rfc-editor.org/rfc/rfc3339

6. Android Open Source Project. UsageStatsManager.
   https://developer.android.com/reference/android/app/usage/UsageStatsManager

7. Android Open Source Project. UsageEvents.
   https://developer.android.com/reference/android/app/usage/UsageEvents

8. Android Open Source Project. Android Debug Bridge (adb).
   https://developer.android.com/tools/adb

9. Android Open Source Project. dumpsys.
https://developer.android.com/tools/dumpsys

10. The Chromium Authors. HistoryDatabase (Chromium source).
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/components/history/core/browser/history_database.h

11. Mozilla. Places. Firefox Source Docs.
https://firefox-source-docs.mozilla.org/browser/places/index.html

12. Google. Google Takeout.
https://takeout.google.com/

13. Topografix. GPX 1.1 schema.
https://www.topografix.com/GPX/1/1/

14. SQLite. The SQLite Database File Format.
https://www.sqlite.org/fileformat2.html

15. log2timeline project. Plaso documentation.
https://plaso.readthedocs.io/en/latest/

16. log2timeline project. Plaso source repository.
https://github.com/log2timeline/plaso

17. Brignoni, A. ALEAPP (Android Logs Events And Protobuf Parser).
https://github.com/abrignoni/ALEAPP